

WHERE IS MY SHIPMENT

A Probabilistic SKU Location Framework

Project Development Report

Ocean Freight Visibility · Digital Supply Chain Portfolio Project

Table of Contents

- 1. The Problem This Project Responds To.....3
- 2. Vision & Core Approach.....3
- 3. Development Progression.....3
- 4. Issues Found & Fixed.....7
- 5. Feature Architecture.....8
- 6. The Confidence Engine.....9
- 7. SKU Prioritization Framework.....10
- 8. Real-World Grounding.....11
- 9. What's Simulated vs. What's Real.....13
- 10. Deliberate Scope Exclusions.....13
- 11. Testing & Verification Methodology.....13
- 12. The Deployed Repository — Real-World Continuation.....15
- 13. Current State Summary.....17
- 14. Future Roadmap.....17
- 15. Conclusion.....18

1. The Problem This Project Responds To

This project did not begin from a specification document. It began from something observed first-hand during an internship at a shipping company: **once a shipment was on the water, it was genuinely difficult to know where a specific SKU actually was during ocean transit.**

The obvious fix — put an RFID tag on every unit — turned out not to be practical. **SKU-level RFID was costly** to deploy at scale: tags, labeling labor, and reader infrastructure add up fast across thousands of units. The alternative already in place, tracking the container itself, wasn't much better: **a single container held many SKUs inside it, so knowing where the container was didn't tell you much about any one specific SKU** — whether it was still there, still sealed, still on schedule.

That gap — too expensive to track every unit directly, not informative enough to track only the container — is **why this project takes a probability-based approach to shipment tracking** instead: instead of physically tracking each SKU, infer a confidence level for it from the logistics events that already happen around it, and give planners a quantified answer instead of a guess.

This is also why the project is scoped to **ocean freight only**, deliberately excluding air, truck, and rail — it reflects the specific environment the problem was observed in, not an oversight.

2. Vision & Core Approach

Instead of adding hardware to every SKU, the system infers each SKU's location and reliability from logistics events that already happen — packing scans, container seals, gate movements, customs holds. Every SKU starts at 100% confidence the moment it is verified packed and sealed inside its container. From there, the confidence score only moves when something logistically meaningful happens to that container or SKU.

This turns a flat status update — *“Container ABC is in Singapore”* — into an actionable, probabilistic one — *“SKU 4589 has a 94% probability of being inside Container ABC and is expected to arrive in Rotterdam in 6 days.”*

Why rule-based, not machine-learned, for version 1

The confidence engine is deliberately a transparent, weighted rule-based model rather than a black-box ML model. Every score change traces back to a named event with a fixed, visible weight, so a planner — or an interviewer — can always ask “why did this drop?” and get a concrete answer (e.g. Customs Inspection Flagged, -16 points) instead of an opaque prediction. The system also includes an illustrative “Calibrated” mode showing what a future Bayesian/ML recalibration might look like — clearly labeled as hypothetical, since no real historical outcome data currently exists to calibrate against.

3. Development Progression

The project was built iteratively across thirteen phases, each triggered by a specific request or a specific problem discovered during review. The phases below are presented in the order they actually happened, including the false starts and corrections — the corrections are as much a part of the development record as the features.

Phase 1 — Initial Prototype

The first build stood up the core loop end to end: a synthetic dataset generator (shipments, containers, SKUs, and a logistics event timeline for each), a weighted confidence-scoring engine matching the original problem framing, and three tabs — a Dashboard with fleet-wide KPIs, a SKU Search view centered on a “custody ladder” timeline visualization, and an Exceptions worklist sorted by lowest confidence. Scale was deliberately reduced from an initial “50,000 SKU” ambition to roughly 500–600 SKUs so the prototype would stay responsive in a browser, with the assumption stated explicitly rather than left implicit.

Phase 2 — Feature Expansion

A Help section was added explaining the problem and the probability-based solution in plain terms. This was followed by a larger expansion: a header alerts bell surfacing Alert-risk SKUs, a What-If Simulator for testing hypothetical events against a real SKU without touching live data, an illustrative Rule-based / Calibrated confidence-mode toggle, and an Integrations tab mocking a live AIS vessel feed and an ERP reconciliation table — both clearly labeled as simulated, since this environment has no route to a real AIS provider or ERP system.

Phase 3 — Rebrand & Layout Fixes

The header navigation began overflowing the visible screen once enough tabs and controls had accumulated. The header was rebuilt as a three-row layout (logo/alerts, a horizontally-scrollable control strip, and a stats line) so extra tabs scroll within their own row instead of spilling past the screen edge. In the same pass the product was renamed from the working title “PSLE (Probabilistic SKU Location Engine)” to

and dead code from an earlier draft (an entire unused duplicate search component, unused imports) was removed.

Phase 4 — Comprehensive Bug Audit

Rather than accept the prototype as finished, it was audited deliberately for shortcomings. Several were found by reading the code; two of the most serious were only found by actually running the data-generation logic in Node and measuring the output rather than inspecting the

code by eye. That distinction — measured versus assumed — became the standard for the rest of the project. The full list of issues found in this phase and later phases is consolidated in Section 4.

Phase 5 — Documentation & Explainability Pass

Every function in the codebase was explained in detail, first in technical terms and then rewritten in plain language for a non-technical reader, at request. This forced a full self-review of the architecture and surfaced no new defects, but confirmed the mental model of the system (data generation → shared UI primitives → per-tab views → the App shell wiring state through all of them) was coherent and easy to narrate.

Phase 6 — Strategic Design Discussion

Before adding more features, five open design questions were raised and answered on their merits rather than immediately built:

- Whether the 95% / 80% risk thresholds from the original framing were **correct** — concluded they were intentionally conservative by design, favoring early warning over waiting for near-certainty, and that this is a business judgment call worth making tunable rather than fixed.
- What real **ERP integration** would require — a backend proxy holding credentials, a real protocol (SAP OData/IDocs or Oracle REST), a field-mapping layer, and a sync strategy.
- Whether **weather/AIS** could be made real — yes in principle, given a real AIS feed, a real marine-weather API, and a model connecting storm severity to actual delay risk; not possible inside this sandboxed environment.
- Whether the **Rule-based / Calibrated toggle** was actually necessary — concluded it added value as a demonstration of the PRD's own v1→v2 roadmap, but only if labeled honestly as illustrative rather than evidence-backed.
- The **reasoning behind each event's point weight** — established the principle that weight should scale with how directly an event proves physical tampering (a broken seal is stronger evidence than a delay).

Phase 7 — Settings & Tunability

Following that discussion, a full Settings tab was built rather than treating the weights and thresholds as fixed: editable risk thresholds, editable point values for every disruption type, and a live preview showing the resulting Clear/Monitor/Alert split update in real time. The Rule-based/Calibrated distinction was clarified with a side-by-side comparison table rather than removed, and the Help tab was renamed “Logic” and rewritten to state plainly, without hedging, that the ERP and weather data are fabricated — and to explain what a real integration would need for each.

Phase 8 — Geopolitical & Compliance Risk

Route-based risk was added for real, named maritime chokepoints — the Strait of Hormuz, the Gulf of Aden / Somali Basin, the Strait of Malacca, and the Red Sea / Bab-el-Mandeb — verified as legitimate shipping-industry corridors rather than assumed. A deliberate design choice was made

- not to hardcode a list of “sanctioned countries” directly, since real sanctions lists are a legal/compliance matter that changes over time and should be owned by a compliance team, not asserted by the tool. Instead, a user-editable

compliance watchlist was built: any port or region a user types in gets flagged, with the tool stating plainly that this is not a legal sanctions determination.

Phase 9 — Scope Discussion & Data-Quality Design

A second round of design questions preceded implementation, again discussed before being built:

- Whether the prototype was “whole” — identified real gaps (cost/ROI framing, data-latency realism, multi-modal coverage, a dollar-value-at-risk metric) as an honest answer rather than a reassuring one.
- Why **ocean-only** — reaffirmed as a defensible, experience-grounded scope decision rather than an oversight (see Section 1).
- Why filter **all SKUs** rather than only high-value ones — proposed value, urgency, and a custom list as a starting filter set, plus SLA tier, already-late status, and perishability; deliberately excluded production-dependency and substitutability as a scoping decision, not an omission.
- Whether **ROI/cost quantification** belonged in this pass — excluded at the requester’s decision.
- What a real **data-quality** layer should contain — proposed a grace window before flagging a missing scan, self-healing reclassification when a SKU reappears later, conflict detection between disagreeing systems, timestamp/logical-order sanity checks, and per-source reliability weighting.

Phase 10 — SKU Prioritization & Data-Quality Implementation

Everything proposed in Phase 9 (excluding ROI, production-dependency, and substitutability, per the scoping decision) was implemented as real, working logic rather than cosmetic UI:

- **Filters:** value, urgency (Low/Medium/High), SLA tier, perishability, already-late status, and a free-text custom list, plus a transparent Priority Score combining all of them for a single ranked worklist.

- **Data quality:** a missing-scan grace window with automatic “self-healing” reclassification, synthetic dual-source conflict detection, real (not seeded) timestamp/logical-order anomaly detection, and per-source reliability weighting that dampens reports from historically noisy sources such as GPS telemetry.

Building the “already late” filter surfaced a genuine structural bug in the underlying model, described in Section 4.

Phase 11 — Real-World Terminology Accuracy Pass

On review, the shipment milestone sequence used generic labels (“Packed,” “Loaded into Container,” “Gate Out”) rather than real industry terminology. This was corrected by verifying actual terms through research rather than relying on memory alone, then rebuilding the full seventeen-step milestone chain with verified language: Booking Confirmed, Container Stuffing Completed, VGM Submitted, Carrier Seal Affixed, Container Gate-In, Customs Export Clearance (LEO Issued), Loaded per Stowage Plan, Bill of Lading Issued, Vessel Departure, Transshipment at Hub Port, Vessel Arrival, Discharged from Vessel, Import Manifest Filed, Customs Import Clearance, Container Gate-Out, Warehouse Received, and Delivered / Empty Returned. India-specific customs terms (LEO, IGM, Bill of Entry) were included alongside their global equivalents rather than presented as universal, and container “barging” — verified as a genuine, documented practice at the Rhine-connected ports of Rotterdam, Antwerp, and Hamburg — was added as a neutral, non-risk custody event specific to those ports. A full glossary explaining every term was added to the Logic tab.

Phase 12 — Simulator UX Redesign

The What-If Simulator’s original design — a grid of buttons that immediately applied an event on click — was judged unclear for a new user. It was rebuilt around an explicit three-step flow: choose a category (Disruption or Recovery), select an event, and press a dedicated Run button to apply it, with the event’s reasoning shown before committing.

Phase 13 — Simulator Multi-Select Refinement

A follow-up question — why the simulator only allowed one event at a time, and whether repeated runs were compounding correctly or resetting — was investigated empirically before being answered. The compounding behavior was verified to already work correctly (each run genuinely stacked on the last). The single-select constraint was agreed to be a real limitation, and the simulator was rebuilt around checkboxes: multiple events (even mixing Disruption and Recovery) can be queued, reviewed as a visible summary with individual reasoning shown for each, and applied together with one Run action — while still supporting further rounds stacked on top afterward. A visible “N applied” counter was added between the two confidence gauges to remove the ambiguity that prompted the question in the first place.

4. Issues Found & Fixed

Several of the issues below were not visible from reading the code — they only surfaced when the data-generation logic was actually run and measured, or when the interface was actually clicked through rather than assumed to work. That distinction is called out explicitly where it applies, since it materially changed how the issue was found.

#	Issue	Impact	Resolution
1	Shipment ETA could fall in the past for a shipment still shown as “in transit,” with no cause attached.	Measured at 12.2% of all shipments — found by running the generator and counting outcomes, not by inspection.	Transit duration reworked so remaining time is always derived forward from departure, guaranteeing a sensible ETA for any undisrupted shipment.
2	A SKU’s description text and its category field were drawn from two independent random picks and frequently disagreed (e.g. description read “Electronics item” while the category badge read “Apparel”).	Measured at 85.8% of all SKUs — the single highest-impact defect found in the project.	Both fields are now derived from one value, eliminating the mismatch entirely (verified at 0% after the fix).
4	Pallet IDs were computed from a global counter modded against a container-local count, which did not cleanly group SKUs into real pallets.	Pallet grouping was effectively meaningless data.	Replaced with an explicit, dedicated pallet counter; verified no pallet spans more than one container.
5	The Exceptions tab’s “All flagged” filter silently included fully healthy (Clear-risk) SKUs.	Misleading worklist — the label implied Monitor + Alert only.	Filter logic corrected to exclude Clear-risk SKUs under “All flagged.”
6	The Exceptions “Latest event” column showed whichever event happened last chronologically, which for a shipment that had since progressed could be a routine event like “Delivered” — hiding the actual reason a SKU was flagged.	A planner scanning the worklist could not tell why a flagged SKU was flagged.	Column redefined to show the most recent event that actually reduced confidence, and relabeled “Flagged for.”
7	SKU Search selection relied on a “force a remount by incrementing a key” workaround, plus a callback prop that was wired in but never actually invoked.	A real, reproducible edge case: re-selecting the same SKU a second time (after navigating elsewhere) could silently fail to update the view, due to React bailing out on a no-op state update.	Replaced with a single controlled selection value owned by the top-level app — structurally cannot drift out of sync.

#	Issue	Impact	Resolution
8	The Dashboard's "Delayed Containers" KPI only counted delay/weather events, undercounting containers actually flagged for customs holds, seal breaks, or GPS anomalies.	Headline metric materially understated real disruption exposure.	Broadened to count any event that reduced confidence, and relabeled "Disrupted Containers."
9	Header/navigation overflowed the visible screen once enough tabs and controls had accumulated.	Tabs ran off-screen and were unreachable on narrower viewports.	Header rebuilt as a three-row layout with a horizontally scrollable control strip.
10	The "already late" shipment flag was mathematically impossible to ever be true: since a delay can only push a future ETA further into the future, an additive model can never make the current best-estimate ETA fall in the past.	The "Already late" filter and "Late Shipments" KPI silently returned zero results no matter how disruption severity was tuned.	Redefined lateness as "accumulated real disruption-driven slip exceeds the shipment's original schedule buffer" — a more defensible definition, verified at ~12% of SKUs after the fix.
11	The Simulator's CURRENT confidence gauge never changes (by design, as a fixed baseline), which could look like repeated Run actions were not compounding.	Risked being mistaken for a bug — investigated directly rather than assumed; the underlying stacking math was confirmed correct via live interactive testing.	Added an explicit "N applied" counter between the two gauges so compounding is visibly obvious, not just inferable.

A note on methodology

Roughly half of the issues above (1, 2, 8, 10, and 11) were found or confirmed by **actually executing the code and measuring the result** — running the data generator in isolation and counting outcomes, or driving the interface with simulated clicks — rather than by reading the source and reasoning about what it should do. Issue 10 in particular would not have been caught by code review alone: the logic read as correct on inspection, and only failed when checked against real generated output across many trials.

5. Feature Architecture

The finished prototype is organized into seven tabs, each serving a distinct role. All of them read from the same underlying confidence engine, so a change made in Settings is reflected consistently everywhere at once.

Dashboard

Fleet-wide health at a glance: active shipment count, average confidence, disrupted-container count, critical-SKU count, shipments transiting known high-risk waters, and late-shipment count, alongside a confidence-by-category chart and a risk-tier breakdown.

SKU Search

Look up a single SKU and see everything about it: current predicted confidence, container/pallet/shipment identifiers, route, ETA (including the originally promised date if it has since slipped), route-based risk exposure, SLA tier, urgency, shelf life if perishable, and its full “custody ladder” — a vertical timeline of every event in its history, each colored by the confidence level immediately after that event.

Exceptions

A prioritized worklist rather than a flat list. Filterable by risk tier, urgency, SLA tier, perishability, already-late status, minimum value, and a free-text custom list; sortable by confidence or by a composite Priority Score. See Section 6 for the scoring detail.

Simulator

A what-if tool for testing hypothetical events against a real SKU without altering any real data — useful for answering “should I expedite a backup shipment?” before committing. Supports queuing multiple Disruption and/or Recovery events together, reviewing them before committing, and running further rounds on top afterward.

Settings

The control surface for the whole confidence engine: editable risk thresholds (Clear / Monitor / Alert boundaries), editable point weights for every disruption type and route-risk corridor, the compliance watchlist, and the full data-quality layer — missing-scan grace window, conflict-detection weight, timestamp-anomaly weight, and per-source reliability multipliers. A live preview shows the resulting risk distribution update in real time as any of these change.

Integrations

An illustrative view of what a live AIS vessel feed and an ERP reconciliation sync would surface once connected to real systems — explicitly labeled as simulated, since this environment cannot reach either a real AIS provider or a real ERP system.

Logic

The documentation layer: the problem and solution in plain language, a full glossary of the real shipment-milestone terminology used throughout the app, a side-by-side table comparing Rule-

based and Calibrated weights with the reasoning behind each, and an explicit, itemized account of exactly what in the prototype is real working logic versus a simulated stand-in — and what a production version of each simulated piece would require.

6. The Confidence Engine

At the center of the system is a single scoring function that replays a SKU’s full event history from a baseline of 100 and adjusts it event by event. Every other view in the app — Dashboard, Search, Exceptions, the Simulator — reads the output of this one function, so a change made in one place is guaranteed to be reflected everywhere else.

Base weighting

Each disruption type carries a default point weight, set to scale with how directly the event proves physical tampering rather than a mere timing issue:

Event	Default weight	Reasoning
Unexpected Delay	-3	Routine, usually resolves on its own — weak evidence anything is wrong with the SKU.
Weather Disruption	-4	Affects timing, not custody — the goods themselves were almost certainly not touched.
GPS Anomaly	-10	A real visibility gap, but often a sensor/connectivity glitch rather than proof of tampering.
Missing Scan	-12	A process failure — a checkpoint was skipped. Moderately concerning on its own.
Customs Inspection Flagged	-16	The container was physically opened and handled by a third party — a meaningfully stronger signal.
Seal Broken	-25	Direct physical evidence of unauthorized access — the strongest signal something may be missing.

Rule-based vs. Calibrated

Both modes replay the identical event history for a SKU — nothing about what happened changes. **Rule-based** mode uses whatever weight is currently set (the table above by default). **Calibrated** mode takes that same starting weight and multiplies it by a fixed adjustment factor — for example, Customs Inspection Flagged is weighted 1.35× up, since calibration would be expected to find it correlates more strongly with real loss than the raw rule assumes, while Weather Disruption is weighted 0.6× down. These multipliers are stated plainly as illustrative,

not derived from real historical outcome data, since none exists yet — the mode exists to demonstrate the v1-to-v2 evolution path the original problem framing described, not to claim evidence it doesn’t have.

Data-quality safeguards

A missing scan is usually a reporting gap, not proof a SKU is actually at risk — treating the two identically was identified as a real weakness and addressed with four safeguards, all tunable in Settings:

- **Grace window + self-healing.** A missing scan under a configurable age (default 24 hours) does not count against confidence yet. If a later checkpoint confirms the SKU reappeared, the gap is automatically reclassified as self-healed with zero lasting impact, rather than remaining a permanent unexplained risk.
- **Conflict detection.** A small share of containers receive a synthetic “two systems disagree” event (e.g. Carrier EDI vs. Terminal System) — a real category of problem, though seeded at a flat rate here rather than arising from genuinely independent live feeds.
- **Timestamp anomaly detection.** This one is not seeded — it is real logic that checks whether a disruption was logged before a milestone it logically requires (a customs hold before export clearance had even started; a broken seal before one was ever affixed), and catches genuine inconsistencies the randomized generator produces.
- **Source reliability weighting.** Every disruption is tagged with the system that reported it, and its weight is multiplied by that source’s trust factor. GPS Telemetry defaults to 0.6×, reflecting that GPS trackers are a known-noisier source than, for example, the Terminal System.

7. SKU Prioritization Framework

In reality, visibility is needed into every SKU, but a planner’s attention is finite — a low-value item and a high-value shipment cannot be treated as equally urgent. The Exceptions worklist filters and ranks on the following signals, layered on top of the existing risk tiers:

Signal	What it captures
Value	A per-SKU dollar figure (quantity × unit price). Higher-value SKUs weigh more heavily in the Priority Score.
Urgency (Low / Medium / High)	Derived automatically, not manually set — driven by already-late status, shelf life remaining on perishables, and how little time is left before the current ETA.
SLA tier	Standard, Priority, or Contractual SLA — a proxy for penalty or relationship exposure if the SKU is late.

Signal	What it captures
Perishable / shelf life	Pharmaceuticals are always flagged perishable; a small share of other categories are too, each with a remaining shelf-life countdown.
Already-late status	Derived directly from the confidence engine, not randomized — see below.
Custom list	A free-text filter matching any SKU ID or customer name fragment, for ad-hoc lookups.

Where “already late” comes from

Delay, weather, and customs-hold events each carry a realistic schedule-slip figure, in days. A shipment is flagged late when its **accumulated** slip exceeds the schedule buffer it started with — meaning known, named disruptions have already consumed the cushion, so it is on track to miss its original promised date even though the displayed ETA (always a sensible future date, never a confusing negative one) has simply moved out to account for it. This definition replaced an earlier, structurally broken version — see Issue 10 in Section 4.

Priority Score

Priority Score = (100 – confidence) + urgency points + SLA points + value points

Urgency points: Low = 0, Medium = 15, High = 30. SLA points: Standard = 0, Priority = 10, Contractual SLA = 20. Value points: 0 / 10 / 20 by value tier. Sorting the Exceptions worklist by this score produces one ranked list instead of juggling several filters independently.

Deliberately out of scope

Two dimensions raised during design discussion were **deliberately excluded**, as a scoping decision rather than an oversight:

- **Production-dependency** — a low-value component that can halt an entire assembly line if missing (impact is not linear with price).
- **Substitutability** — how easily a SKU can be re-sourced or replaced on short notice.

Both are real considerations in a production system and are listed as future work in Section 11, rather than silently omitted.

8. Real-World Grounding

A recurring theme across the project was replacing generic, plausible-sounding placeholders with terminology and mechanics verified against real sources, rather than assumed from memory.

Shipment milestone terminology

The original event chain used generic labels (“Packed,” “Loaded into Container,” “Gate Out”). These were replaced with a verified seventeen-step sequence reflecting real ocean export/import milestones, including terms that differ between global and India-specific customs processes:

Term	Meaning
VGM (Verified Gross Mass)	A mandatory SOLAS safety declaration of a container’s total weight, required before it can be loaded — not optional paperwork.
LEO (Let Export Order)	The Indian customs term for export release; other countries use their own systems (e.g. the US’s AES).
Stowage Plan / Bay Plan	The vessel’s loading plan specifying exactly which bay, row, and tier a container is craned into.
Bill of Lading (B/L)	The carrier’s contract of carriage and title document. A Master B/L covers the whole shipment; a House B/L may be issued by a freight forwarder to the shipper.
IGM (Import General Manifest)	The Indian term for the arrival manifest a carrier files with customs before cargo can be released.
Bill of Entry	The Indian import-clearance filing, made by the importer or their customs broker against the IGM.
Container Barging	Moving containers between a deep-sea terminal and an inland terminal by barge rather than truck or rail — verified as a genuine, routine practice at the Rhine-connected ports of Rotterdam, Antwerp, and Hamburg.

Route & geopolitical risk

Four real, named maritime corridors were added as route-risk zones, verified as legitimate shipping-industry chokepoints and piracy corridors before being included:

- Strait of Hormuz — chokepoint / regional tension
- Gulf of Aden / Somali Basin — established piracy corridor
- Strait of Malacca — the world’s busiest strait and a long-standing piracy hotspot
- Red Sea / Bab-el-Mandeb — has seen militia attacks on commercial shipping during periods of regional conflict

A shipment is flagged as plausibly transiting a zone based on whether its origin or destination port sits near it — stated explicitly as a simplified proxy for real route data, not a substitute for it.

Compliance watchlist — a deliberate design choice

Rather than hardcoding a list of “sanctioned countries,” the system implements a **user-editable watchlist**. This was a deliberate decision, not a shortcut: real sanctions lists (OFAC, EU, UN) change over time and are a legal/compliance matter that should be owned and sourced by a company’s own compliance function, not asserted by a prototype. The tool states plainly, in-app, that the watchlist is not a legal sanctions or denied-party determination.

9. What's Simulated vs. What's Real

Every part of the confidence engine described in Section 6 is real, working logic running on synthetic data — the scoring, the thresholds, the calibration math, the data-quality safeguards, and the priority ranking all genuinely execute. Four things, however, are explicit stand-ins for systems this environment cannot reach, and are labeled as such directly in the product rather than left ambiguous:

Feature	What's simulated	What a real version needs
ERP reconciliation	Material numbers, plants, sales orders, and sync status are invented the moment the tab opens. Nothing is read from or written to a real ERP.	A backend holding credentials (a browser cannot safely call SAP/Oracle APIs directly), a real protocol (SAP OData/IDocs or Oracle Fusion REST), a field-mapping layer, and a sync strategy.
Live AIS vessel feed	A timer invents a new fake vessel position every few seconds.	A real AIS data provider (e.g. a commercial feed or a direct AIS receiver) in place of the timer.
Weather disruptions	A flat, fixed-probability chance per container with a fixed penalty — unconnected to real weather or the vessel’s actual route.	A real vessel position feed, a real marine-weather API queried along the actual route, and a model connecting storm severity to real delay/damage risk.
Route-zone transit	Inferred from origin/destination port proximity, not an actual sailed route.	Real AIS waypoint data to reconstruct the route actually sailed, plus a live maritime-security advisory feed for which corridors are currently high-risk.

10. Deliberate Scope Exclusions

Several ideas were discussed and consciously left out, rather than overlooked. Documenting them here is meant to make the scoping reasoning visible, not just the resulting feature list.

- **Cost/ROI quantification** (e.g. RFID tagging cost vs. expediting cost avoided) — excluded at the requester’s explicit decision, to keep the project focused on the visibility problem rather than a full business case.
- **Production-dependency and substitutability** as prioritization filters — both real considerations, both excluded from this pass by decision (see Section 7).
- **Multi-modal coverage** (air, truck, rail) — out of scope by the same reasoning that grounds Section 1: the problem was observed specifically in an ocean-freight context, and the project stays honest to that origin rather than generalizing beyond what was actually seen.
- **A hardcoded sanctions/denied-party list** — deliberately replaced with a user-owned watchlist, for the reasons given in Section 8.

11. Testing & Verification Methodology

A consistent principle across the project was to verify claims by execution rather than by inspection alone. Several real defects (Section 4) would not have been caught by reading the code and reasoning about what it should do — they were only found by actually running it and measuring the output. The verification approach evolved into four layers, applied repeatedly after each significant change:

1. **Structural checks.** Brace/parenthesis balance and a full bundler pass (esbuild) after every edit, catching syntax errors immediately rather than at first use.
2. **Real render tests.** Every one of the seven tabs rendered with the actual React, Recharts, and icon libraries — not just syntax-checked — using a server-side render pass, to catch runtime errors that only occur once real hooks and real data are involved.
3. **Direct execution and measurement.** The data-generation function was run in isolation and its output measured directly — counting mismatches, checking distributions, verifying location assignments — rather than trusting that the logic “looked right.” This is how the ETA bug, the category-mismatch bug, and the structural lateness bug were actually found and confirmed fixed.
4. **Interactive DOM testing.** For the Simulator’s multi-select rework, a real DOM environment with simulated user clicks confirmed the entire flow end to end — checking two events, seeing the queue summary and reasoning text appear, running them together, confirming the exact combined point drop, running a second round, and confirming further compounding rather than a reset.

This approach was applied consistently, including immediately before answering the question “is everything done?” late in the project — rather than asserting completeness, a full render pass across all seven tabs plus an end-to-end check (tightening thresholds only ever increases the

Alert count; adding a watchlist entry visibly changes real confidence values) was run first, and the answer was based on those results.

12. The Deployed Repository — Real-World Continuation

Everything above documents how the prototype was designed and built collaboratively.

Separately, the underlying concept was carried forward into a real, standalone repository — github.com/Shubham-Gambhire/whereismyshipment — and iterated on independently using **Lovable**, an AI pair-programming tool, across 160 commits over three days.

This section documents that repository directly, based on its actual commit history and source code rather than a description of it.

Confirmed lineage

The repository’s core logic file, `src/components/prototype/WhereIsMyShipment.tsx` (3,380 lines), is a direct descendant of the JSX prototype built in this project. This isn’t inferred from similarity — it’s confirmed structurally: the file contains functions with identical names and identical responsibilities to the ones documented in Sections 5–6 of this report, including `generateData`, `computeTimeline`, `riskFromConfidence`, `DashboardView`, `ExceptionsView`, `WhatIfView`, `SettingsView`, `IntegrationsView`, `AlertsBell`, `LogicView`, `SearchView`, `CustodyLadder`, and `ConfidenceGauge`. The prototype was carried into a production-shaped project scaffold and then substantially extended.

Technology stack

The repository runs on a materially more production-oriented stack than the single-file JSX prototype:

- **Framework:** TanStack Start (React 19) with TanStack Router and TanStack Query, built on Vite.
- **UI system:** Tailwind CSS v4 plus a full shadcn/ui component set (Radix primitives) — dialogs, dropdowns, tooltips, sliders, command palette, and more — available even where the prototype doesn’t yet use them all.
- **Data & forms:** Recharts for charting (unchanged from the prototype), React Hook Form with Zod for validation, date-fns for date handling.
- **Tooling:** TypeScript, ESLint, Prettier, and Bun as the package manager.

Development method

Every one of the 160 commits is authored by `gpt-engineer-app[bot]`, Lovable’s automated commit identity, with the project owner recorded as co-author on each — confirming the repository was built through natural-language prompts to an AI agent rather than hand-written commit by commit. The great majority of commits carry the generic message “*Changes*,” which is Lovable’s default for incremental edits; twenty-one commits carry a specific, human-written summary and

mark the real milestones in the project’s growth, shown below with the actual line count of the core prototype file at each point, taken directly from the repository’s history:

Date	Milestone commit	Lines in core file
Aug 1	Initial TanStack Start scaffold created	—
Aug 2	Added base styles and sections	—
Aug 2	Applied portfolio UX changes	2,625
Aug 2	Removed showcase walkthrough	2,624
Aug 2	Showcase removed, root fixed	2,625
Aug 2	Fixed tab layout and styles	2,627
Aug 2	Renamed reset to DEFAULT	2,627
Aug 2	Added control tower dashboard	2,907
Aug 2	Redesigned enterprise layout	3,024
Aug 2	Reworked dashboard UI	3,160
Aug 2	Refined overview branding	3,227
Aug 2	Moved scoring, added logo	3,241
Aug 2	Fixed simulator & filters UI	3,256
Aug 3	Fixed SKU/Simulator pickers	3,283
Aug 3	Update site info for publish	3,283
Aug 3	Add project README	3,283
Aug 3	Added floating SKU bar	3,348
Aug 3	Fixed SKU bar visibility	3,348
Aug 3	Fixed SKU pinning & added variety (current)	3,380

What was genuinely added beyond the conversation prototype

Reading the current source directly, rather than the commit messages alone, surfaces real functional additions that go beyond what this report’s earlier sections describe:

- **Evidence-state vocabulary on the custody ladder.** Each timeline entry is now classified as VERIFIED, EXCEPTION, DELAYED, RESOLVED, or PREDICTED (via a new `custodyState()` function) instead of only being colored by risk tier — a more analyst-facing framing of the same underlying data.

- **Operator notes.** A new `operatorNote()` function generates a short, contextual, plain-spoken assessment for a SKU based on its worst cause (e.g. *“No single clear cause. Evidence has just thinned out across the chain. Treat the date as unsafe,”* or *“Nothing dramatic here. Watch it for another 24h before doing anything”*) — a decision-support layer beyond the structured info cells in the original prototype.
- **A real geographic route map.** A new `RouteMap` component plots shipment lanes using actual port coordinates, deliberately keeping a handful of healthy lanes alongside the flagged ones *“so the map reads as a fleet picture rather than a wall of red”* — a real UX judgment call, not just a technical addition.
- **A plain-language glossary layer.** A new, separate `Onboarding.tsx` component maintains a jargon-to-plain-English glossary (VGM, LEO, Bill of Lading, Stowage Plan, Transshipment, and more) so, in its own words, *“a recruiter, student, or non-logistics visitor can read the same screens an industry expert reads”* — without altering the underlying model.
- **A “control tower” dashboard redesign and rebrand.** The Dashboard was reworked twice (“Added control tower dashboard,” then “Reworked dashboard UI”) and the navigation was renamed for a more product-like feel: Dashboard → *Overview*, SKU Search → *SKU*, Settings → *Model*, Integrations → *Feeds*, alongside a new logo mark and a floating, pinnable SKU status bar.

Consistent honesty positioning

The repository includes a written plan for its own portfolio-facing presentation page, and it explicitly instructs the opposite of oversell: *“Framing throughout: consulting mindset first ... Language is ‘prototype’, ‘concept’, ‘model’ — never ‘case study’ or ‘product launch’.*” That instruction is consistent with the transparency principle carried through the rest of this project (Section 9, *What’s Simulated vs. What’s Real*) — the same discipline about not overstating what the system actually is was applied independently on the deployment side.

Live deployment

The application is deployed and publicly reachable at whereismyshipment.lovable.app, with the source at github.com/Shubham-Gambhire/whereismyshipment.

13. Current State Summary

The table below covers the collaboratively built JSX prototype documented in Sections 1–11. The deployed repository (Section 12) has since carried several of these further — noted inline where applicable.

Tab	Status
Dashboard	Complete — six live KPIs, two charts, shipments table. Independently redesigned twice in the deployed repo (“control tower dashboard,” then a full rework) and renamed “Overview.”
SKU Search	Complete — full detail view, custody ladder, route/SLA/urgency/perishability info. The deployed repo added evidence-state labels (VERIFIED/EXCEPTION/DELAYED/RESOLVED/PREDICTED) and a floating, pinnable SKU bar.
Exceptions	Complete — filters, custom list, Priority Score sort, corrected “all flagged” and “flagged for” logic.
Simulator	Complete — multi-select queue, batch run, repeat-stacking, applied-count clarity.
Settings	Complete — thresholds, event weights, route/compliance weights, watchlist, data-quality controls. Renamed “Model” in the deployed repo.
Integrations	Complete, explicitly illustrative — AIS/ERP clearly labeled as simulated. Renamed “Feeds” in the deployed repo.
Logic	Complete — glossary, weight/mode comparison, simulated-vs-real disclosure, navigation guide. The deployed repo added a separate plain-language onboarding/glossary layer on top of it.

14. Future Roadmap

Items intentionally deferred, in the order they would likely add the most value. Status reflects both the collaborative prototype and the deployed repository as of this report.

5. **Replace the illustrative Calibrated-mode multipliers** with weights genuinely tuned against real historical outcome data, once such data exists. *Not yet started in either codebase.*
6. **Build the real ERP and AIS integrations** described in Section 9, starting with a backend proxy layer. *Not yet started in either codebase — both remain explicitly simulated.*
7. **Add production-dependency and substitutability** as additional prioritization dimensions. *Not present in either codebase.*
8. **Introduce a cost/ROI framing** — value at risk, expediting cost avoided — once the visibility problem itself is validated. *Not present in either codebase.*
9. **Evaluate whether the underlying confidence-engine pattern generalizes to other transport modes**, without assuming it does. *Not evaluated.*
10. **Give the deployed repository’s new additions** (operator notes, evidence-state labels, the route map, the onboarding glossary) the same scrutiny this report applied to the original

prototype — they have not yet been audited or empirically tested the way Sections 4 and 11 describe.

15. Conclusion

The system that exists today is meaningfully different from the first working prototype — not just in feature count, but in how many of its numbers are now defensible rather than assumed. Real industry terminology replaced generic placeholders. A structurally broken lateness definition was found and corrected. Nearly nine in ten SKUs once carried a self-contradictory description, silently, until the data was actually measured rather than trusted. None of that would have surfaced from a feature checklist alone.

The project also continued past this conversation: the same prototype was carried into a real, deployed repository and extended independently — a control-tower dashboard redesign, evidence-state framing on the custody ladder, operator-style decision notes, a geographic route map, and a plain-language onboarding layer, all verifiable directly in github.com/Shubham-Gambhire/whereismyshipment. That continuity, confirmed by matching function names and logic rather than assumed from similarity, is itself evidence that the thinking behind the prototype held up outside the conversation that produced it.

The report documents its own boundaries deliberately throughout: what is real logic versus a labeled stand-in, what was left out on purpose, and what has been built since but not yet re-audited — rather than presenting a single finished snapshot. That distinction is treated here as part of the deliverable, not a gap to hide.